

Doskonale! Wchodzimy głębiej w konkrety techniczne i projektowe dla TipJar. Rozłożmy to na części:

1. Definicja Specyfikacji (Funkcjonalne i Niefunkcjonalne)

Specyfikacje Funkcjonalne (Co system ma robić):

- **Zarządzanie Użytkownikami (Twórcy):**

- Rejestracja: Możliwość założenia konta (np. przez e-mail/hasło lub powiązanie z kontem Google/Twitch - do rozważenia). Kluczowe jest powiązanie konta TipJar z nowo utworzonym lub istniejącym portfelem Circle (developer-controlled).
- Logowanie: Bezpieczny dostęp do konta.
- Profil Twórcy: Możliwość stworzenia i edycji profilu (@username unikalny, wyświetlana nazwa, awatar, krótki opis, linki do mediów społecznościowych).
- Generowanie Linku/QR: Automatyczne tworzenie unikalnego linku (tipjar.com/@username) i kodu QR kierującego do strony profilu.
- Panel (Dashboard): Przegląd całkowitej sumy otrzymanych napiwków (w USDC), lista ostatnich transakcji (kwota, data, nadawca - z opcją anonimowości, wiadomość), status ustawionych celów.
- Zarządzanie Celami (Opcjonalne): Możliwość zdefiniowania celu zbiórki (np. "Na nowy mikrofon") z kwotą docelową i widocznym postępem.
- Konfiguracja Odbioru: Wybór preferowanej sieci L2 (np. Polygon, Solana, Arbitrum) do otrzymywania środków na powiązanym portfelu Circle.
- Wyплаты: Inicjowanie transferu USDC z portfela Circle powiązanego z TipJar na zewnętrzny, własny portfel krypto twórcy. (W przyszłości potencjalna integracja z Circle off-ramp do wypłaty fiat).

- **Proces Wsparcia (Fani):**

- Dostęp do Profilu Twórcy: Możliwość otwarcia strony profilu przez link lub kod QR.
- Widok Profilu: Wyświetlanie informacji o twórcy (awatar, nazwa, opis), ewentualnego celu zbiórki i jego postępu.
- Interfejs Napiwku:
 - Wybór Kwoty: Intuicyjny wybór kwoty (np. predefiniowane przyciski: \$1, \$5, \$10; suwak; pole do wpisania własnej kwoty).
 - Wiadomość (Opcjonalnie): Pole tekstowe do dodania krótkiej wiadomości dla twórcy.
 - Anonimowość: Opcja (checkbox) wysłania napiwku anonimowo (nie wyświetla nazwy fana twórcy).
- Wybór Metody Płatności:
 - Płatność Wewnętrzna (Preferowana): Użycie USDC z portfela Circle powiązanego z kontem fana w TipJar (jeśli fan zdecyduje się je założyć dla wygody).
 - Płatność Zewnętrznym Portfelem: Możliwość połączenia portfela typu MetaMask i wysłania USDC (wymaga obsługi połączenia i monitorowania transakcji z zewnątrz - bardziej złożone UX).
 - Płatność Kartą (Integracja Fiat-on-Ramp): Możliwość zapłaty kartą, gdzie system w tle kupuje USDC i przekazuje je twórcy (wymaga partnera do obsługi fiat-on-ramp).
- Realizacja Płatności: Przeprowadzenie transakcji USDC na wybranej sieci L2, z opłatami za gaz pokrywanymi przez platformę TipJar (via Circle Gas Station/Paymaster).
- Potwierdzenie: Wyraźne potwierdzenie pomyślnego wysłania napiwku.

- **Funkcje Platformy:**

- Zarządzanie Portfelami Circle: Bezpieczne tworzenie i zarządzanie portfelami (developer-controlled) dla twórców i (opcjonalnie) fanów za pomocą Circle Wallet API.
- Orkiestracja Transakcji: Inicjowanie i monitorowanie transferów USDC między portfelami na L2 za pomocą Circle API.
- Sponsorowanie Gazu: Integracja z Circle Gas Station/Paymaster API w celu pokrywania opłat transakcyjnych.
- Przechowywanie Danych: Bezpieczne przechowywanie danych użytkowników, profili, historii transakcji (dane off-chain dla szybkiego dostępu).
- (Przyszłość) Obsługa Subskrypcji: Mechanizm cyklicznych płatności USDC.
- (Przyszłość) Konwersja Fiat-USDC: Integracja z partnerem obsługującym płatności kartą i wymianę na USDC.

Specyfikacje Niefunkcjonalne (Jak system ma działać):

- **Bezpieczeństwo:** Najwyższy priorytet. Ochrona danych użytkowników (zgodność z RODO/GDPR), bezpieczne zarządzanie kluczami (wykorzystanie MPC w Circle Wallets), ochrona przed atakami webowymi (OWASP Top 10), bezpieczna komunikacja API.
- **Wydajność:** Szybkie ładowanie stron (<2s), czas potwierdzenia transakcji zależny od L2 (sekundy/minuty), ale interfejs użytkownika musi reagować natychmiastowo. Niskie opóźnienia API.
- **Skalowalność:** Architektura umożliwiająca horyzontalne skalowanie komponentów (backend, baza danych) w odpowiedzi na rosnące obciążenie.
- **Użyteczność (UX):** Krytyczna. Interfejs musi być ekstremalnie prosty, intuicyjny i przyjazny nawet dla osób bez doświadczenia w krypto. Minimalna liczba kroków do wysłania/otrzymania napiwku.
- **Niezawodność:** Wysoka dostępność usług (np. 99.9% uptime), odporność na błędy, mechanizmy monitorowania i logowania transakcji.
- **Utrzymywalność:** Czysty, dobrze udokumentowany kod, modularna architektura, zautomatyzowane testy.
- **Kompatybilność:** Wsparcie dla najnowszych wersji głównych przeglądarek (Chrome, Firefox, Safari, Edge). Projekt responsywny (desktop, tablet, mobile).

2. Wybór Technologii

- **Frontend:**

- **Framework: Next.js (React)** - Dojrzały, wydajny framework z renderowaniem po stronie serwera (SSR) i statycznym generowaniem stron (SSG), co poprawia SEO i szybkość ładowania. Duża społeczność i ekosystem.
- **Język: TypeScript** - Dla bezpieczeństwa typów i lepszej organizacji kodu.
- **Styling: Tailwind CSS** - Utility-first CSS framework umożliwiający szybkie budowanie customowych interfejsów bez opuszczania HTML/JSEX. Zapewnia spójność i łatwość utrzymania stylów.
- **Zarządzanie Stanem: Zustand** lub **Jotai** - Lżejsze alternatywy dla Reduxa, wystarczające dla większości potrzeb zarządzania stanem globalnym w aplikacji.
- **Interakcja z Blockchain (jeśli potrzebna poza Circle): Viem** - Nowoczesna, lekka i wydajna biblioteka do interakcji z Ethereum i sieciami EVM-kompatybilnymi (jak Polygon).

- **Backend:**

- **Framework: NestJS (Node.js)** - Progresywny framework Node.js zbudowany na TypeScript. Wymusza dobrą architekturę (moduły, kontrolery, serwisy), ułatwia

- skalowanie i utrzymanie. Dobry do budowy API REST/GraphQL.
 - **Język: TypeScript** - Spójność z frontendem, bezpieczeństwo typów.
 - **API: REST** - Standardowy i dobrze rozumiany wybór dla komunikacji frontend-backend. GraphQL można rozważyć w przyszłości.
- **Baza Danych:**
 - **System: PostgreSQL** - Niezawodna, relacyjna baza danych, dobrze radząca sobie ze strukturalnymi danymi (użytkownicy, profile, transakcje). Obsługuje transakcje ACID.
 - **ORM: Prisma** - Nowoczesny ORM dla Node.js/TypeScript, ułatwiający interakcję z bazą danych, migracje schematu i zapewniający bezpieczeństwo typów.
- **Integracja Blockchain/Płatności:**
 - **Rdzeń: Circle API** (Programmable Wallets API, Gas Station API, Paymaster API, Payments API - dla fiat, CCTP API - dla cross-chain) - Kluczowy element, abstrakcja nad bezpośrednią interakcją z blockchainem dla głównych przepływów.
- **Infrastruktura/DevOps:**
 - **Hosting Frontend: Vercel** - Idealnie zoptymalizowany pod Next.js, oferuje łatwe deploymenty, CI/CD, podglądy gałęzi.
 - **Hosting Backend/DB: AWS** (np. ECS/EKS dla kontenerów, RDS dla PostgreSQL) lub **Google Cloud** (Cloud Run/GKE, Cloud SQL) lub **Render** (prostsza alternatywa Platform-as-a-Service).
 - **Konteneryzacja: Docker** - Do tworzenia spójnych środowisk deweloperskich, stagingowych i produkcyjnych.
 - **CI/CD: GitHub Actions** (jeśli kod jest na GitHubie) lub **GitLab CI**.

3. Projekt Architektury Systemu i UX/UI

Architektura Systemu (Wysokopoziomowo):

graph LR

```

    subgraph Użytkownik
        Browser[Przeglądarka Fana/Twórcy]
    end

```

```

    subgraph Frontend (Vercel)
        NextApp[Aplikacja Next.js]
    end

```

```

    subgraph Backend (AWS/GCP/Render)
        ApiGw[API Gateway] --> BackendApi[Backend API (NestJS)]
        BackendApi --> Db[(PostgreSQL / Prisma)]
        BackendApi --> CircleApi[Circle API Gateway]
    end

```

```

    subgraph Circle Platform
        CircleApi --> ProgWallets[Programmable Wallets]
        CircleApi --> GasStation[Gas Station/Paymaster]
        CircleApi --> PaymentsApi[Payments API (Fiat)]
        CircleApi --> CCTP[CCTP API]
    end

```

```

    subgraph Blockchain

```

```

    L2Network[Sieć L2 (np. Polygon)]
end

Browser -- HTTPS --> NextApp
NextApp -- REST API --> ApiGw
ProgWallets -- Transakcje --> L2Network
PaymentsApi -- Interakcje --> Dostawca Fiat On-Ramp

style Circle Platform fill:#D6EAF8,stroke:#333,stroke-width:2px
style Blockchain fill:#E8DAEF,stroke:#333,stroke-width:2px

```

Przepływ Danych (Przykład - Napiwek od Fana):

1. **Fan (Browser):** Wchodzi na tipjar.com/@tworca. Przeglądarka pobiera stronę z **NextApp (Frontend)**.
2. **NextApp:** Pobiera dane profilu twórcy, wysyłając zapytanie do **BackendApi**.
3. **BackendApi:** Pobiera dane twórcy z **Db (PostgreSQL)** i zwraca do **NextApp**.
4. **Fan (Browser):** Wpisuje kwotę \$5, wiadomość, klika "Wyślij Napiwek".
5. **NextApp:** Wysyła żądanie (kwota, wiadomość, ID twórcy, dane uwierzytelniające fana) do **BackendApi**.
6. **BackendApi:**
 - Waliduje dane.
 - Identyfikuje portfel Circle fana i twórcy.
 - Wysyła żądanie do **CircleApi (Programmable Wallets)**: "Przelej 5 USDC z portfela Fana na portfel Twórcy na sieci Polygon".
 - W żądaniu do Circle wskazuje użycie **Gas Station/Paymaster** do pokrycia opłat.
7. **Circle Platform:**
 - Używa MPC do autoryzacji transakcji.
 - Wysyła transakcję do **L2Network (Polygon)**.
 - Monitoruje status transakcji.
8. **L2Network:** Potwierdza transakcję.
9. **Circle Platform:** Otrzymuje potwierdzenie i powiadomienia (np. przez webhook) **BackendApi**.
10. **BackendApi:** Zapisuje transakcję w **Db**, aktualizuje saldo twórcy, powiadamia **NextApp** o sukcesie.
11. **NextApp:** Wyświetla fanowi potwierdzenie. Dashboard twórcy zostanie zaktualizowany przy następnym odświeżeniu/przez WebSocket.

Szczegółowe Projektowanie UX/UI (Opis kluczowych ekranów/komponentów):

- **Strona Główna (Marketingowa):** Czysta, nowoczesna. Nagłówek wyjaśniający korzyści (niskie prowizje, globalne napiwki, USDC). Krótkie sekcje "Jak to działa" dla twórców i fanów. Wyraźne przyciski "Zarejestruj się jako Twórca" i "Przeglądaj Twórców" (jeśli będzie taka funkcja). Stopka z linkami (O nas, Prywatność, Warunki).
- **Rejestracja/Logowanie Twórcy:**
 - Minimalistyczny formularz: E-mail, Hasło lub przyciski "Kontynuuj z Google/Twitch".
 - Po rejestracji: Krok powiązania/utworzenia portfela Circle (jasne instrukcje, dlaczego jest to potrzebne).
 - Pierwsze logowanie: Przekierowanie do kreatora profilu.
- **Kreator Profilu Twórcy:**

- Krok 1: Wybierz unikalny @username.
- Krok 2: Podaj wyświetlaną nazwę, załaduj awatar (kwadratowy/okrągły podgląd), napisz krótki opis (limit znaków).
- Krok 3 (Opcjonalny): Dodaj linki do social mediów (ikony popularnych platform).
- Zakończenie: Wyświetlenie gotowego linku tipjar.com/@username i kodu QR z opcją "Kopiu" / "Pobierz". Gratulacje!
- **Dashboard Twórcy:**
 - Górna belka: Logo TipJar, Nazwa twórcy, rozwijane menu (Edytuj profil, Ustawienia, Wyloguj).
 - Główna sekcja: Duże, wyraźne pole z napisem "Całkowite saldo" i kwotą w USDC (np. \$ 123.45 USDC). Pod spodem mniejszy napis "Dostępne do wypłaty".
 - Sekcja "Ostatnie Napiwki": Tabela lub lista kart. Każdy wpis: Awatar fana (lub ikona anon), Nazwa fana (lub "Anonimowy"), Kwota (+USDC), Wiadomość (skrótowa, klikalna), Data/Godzina. Paginacja lub "Załaduj więcej".
 - Sekcja "Cele" (jeśli aktywna): Pasek postępu z etykietą celu (np. "Nowy Mikrofon: \$350 / \$1000"). Przycisk "Zarządzaj celami".
 - Przycisk "Wypłać środki" w widocznym miejscu.
- **Publiczna Strona Profilu Twórcy (Widok Fana):**
 - Nagłówek: Duży awatar twórcy, Nazwa wyświetlana, @username. Krótki opis pod spodem.
 - Cel (jeśli aktywny): Widoczny pasek postępu celu.
 - **Komponent Napiwku (Kluczowy!):**
 - Tytuł: "Wesprzyj [Nazwa Twórcy]" lub podobny.
 - Wybór Kwoty: Przyciski [\$1] [\$5] [\$10] [\$25]. Suwak (np. od \$1 do \$100 z krokiem \$1). Pole tekstowe "Inna kwota". Wybrana kwota wyraźnie podświetlona.
 - Pole "Wiadomość (opcjonalnie)": Proste pole tekstowe.
 - Checkbox "[] Wyślij anonimowo".
 - Sekcja Płatności (dynamiczna):
 - Jeśli fan zalogowany i ma portfel TipJar: "Zapłać z Twojego portfela TipJar (Saldo: \$X.XX USDC)".
 - Jeśli nie: Przyciski "Zapłać Kartą" / "Połącz Portfel Crypto".
 - Informacja o opłatach: "Opłata transakcyjna (gas): Pokrywana przez TipJar ✨".
 - Główny Przycisk CTA: Duży, wyraźny, np. "Wyślij \$5 Napiwku". Przycisk nieaktywny, dopóki kwota nie jest > 0.
- **Modal Płatności Kartą:** Prosty formularz dostarczony przez partnera fiat-on-ramp (Stripe, MoonPay itp.), osadzony w modalu. Wyraźnie pokazana kwota fiat i szacowana kwota USDC do otrzymania przez twórcę.
- **Modal Potwierdzenia:** Duża ikona sukcesu (np. ✓ lub 🎉). Tekst "Napiwek wysłany! Wysłałeś \$X USDC do [Nazwa Twórcy]". Opcjonalnie przycisk "Udostępnij" lub "Zamknij".

4. Tutorial Konfiguracji Narzędzi Deweloperskich i Środowisk

Założenia: Używamy Git, GitHub, Node.js (LTS), Yarn (lub npm), Docker, VS Code, Next.js, NestJS, PostgreSQL, Prisma, Circle Sandbox.

Krok 1: Instalacja Podstawowych Narzędzi

```
# Zainstaluj Node.js (LTS) - najlepiej przez nvm (Node Version Manager)
```

```
# https://github.com/nvm-sh/nvm
```

```
curl -o-
https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc # lub ~/.zshrc
nvm install --lts
nvm use --lts
```

```
# Zainstaluj Yarn (opcjonalnie, można używać npm)
npm install --global yarn
```

```
# Zainstaluj Git
# Na Ubuntu/Debian: sudo apt update && sudo apt install git
# Na macOS (z Homebrew): brew install git
# Na Windows: https://git-scm.com/download/win
```

```
# Zainstaluj Docker i Docker Compose
# Postępuj zgodnie z oficjalną instrukcją dla Twojego systemu:
https://docs.docker.com/engine/install/
```

Krok 2: Klonowanie Repozytorium i Struktura Projektu

```
# Załóż repozytorium na GitHub/GitLab
git clone <adres_twojego_repozytorium> tipjar-project
cd tipjar-project

# Stwórz strukturę mono-repo (zalecane) lub oddzielne repozytoria
# Przykład mono-repo:
mkdir packages
cd packages
# Utwórz frontend
npx create-next-app@latest frontend --ts --eslint --tailwind --src-dir
--app --import-alias "@/*"
# Utwórz backend
npm install -g @nestjs/cli
nest new backend
cd ../ # Wróć do głównego katalogu projektu
```

Krok 3: Konfiguracja Frontendu (Next.js)

```
cd packages/frontend

# Zainstaluj dodatkowe zależności
yarn add zustand viem # lub npm install ...

# Stwórz plik .env.local (NIE dodawaj go do Git!)
echo "NEXT_PUBLIC_BACKEND_API_URL=http://localhost:3001/api" >
.env.local
# Dodaj inne potrzebne zmienne publiczne (np. klucze API dla usług
frontowych)

# Uruchomienie deweloperskie
```

```
yarn dev # lub npm run dev
# Dostępne pod http://localhost:3000
```

Krok 4: Konfiguracja Backendu (NestJS)

```
cd packages/backend

# Zainstaluj zależności Prisma i NestJS config
yarn add @prisma/client @nestjs/config # lub npm install ...
yarn add -D prisma # lub npm install -D ...

# Inicjalizacja Prisma
npx prisma init --datasource-provider postgresql

# Edytuj schema.prisma (zdefiniuj modele User, Profile, Tip, Goal itp.)
# Przykład w schema.prisma:
# datasource db {
#   provider = "postgresql"
#   url       = env("DATABASE_URL")
# }
# generator client {
#   provider = "prisma-client-js"
# }
# model User { ... }

# Stwórz plik .env (NIE dodawaj go do Git!)
echo
"DATABASE_URL=postgresql://user:password@localhost:5432/tipjar_dev?schema=public" > .env
echo "CIRCLE_API_KEY=twoj_sandbox_api_key" >> .env
echo "CIRCLE_WALLET_SET_ID=twoj_wallet_set_id" >> .env
# Dodaj inne sekrety (JWT_SECRET itp.)

# Uruchom lokalną bazę danych PostgreSQL (używając Dockera)
cd ../../ # Wróć do głównego katalogu projektu
docker run --name tipjar-db -e POSTGRES_USER=user -e
POSTGRES_PASSWORD=password -e POSTGRES_DB=tipjar_dev -p 5432:5432 -d
postgres:15
cd packages/backend

# Wykonaj migrację bazy danych
npx prisma migrate dev --name init

# Wygeneruj klienta Prisma
npx prisma generate

# Skonfiguruj moduł ConfigModule w app.module.ts, aby wczytywał .env
# Skonfiguruj moduł Prisma (zgodnie z dokumentacją Prisma i NestJS)
```

```
# Uruchomienie deweloperskie
yarn start:dev # lub npm run start:dev
# API dostępne pod http://localhost:3001 (lub port skonfigurowany w
main.ts)
```

Krok 5: Konfiguracja Circle Sandbox

1. Zarejestruj się na <https://console.circle.com/>.
2. Przejdź do trybu Sandbox (zwykle przełącznik w interfejsie).
3. Utwórz nową aplikację API, aby uzyskać API Key.
4. W sekcji Programmable Wallets utwórz Wallet Set, aby uzyskać Wallet Set ID.
5. Skonfiguruj uzyskane klucze w pliku .env backendu.
6. Zapoznaj się z dokumentacją Circle API: <https://developers.circle.com/>.

Krok 6: Środowiska Staging i Production

- **Staging:**
 - Cel: Testowanie przed wdrożeniem na produkcję.
 - Konfiguracja: Oddzielna gałąź w Git (np. staging lub develop). Oddzielne instancje aplikacji i bazy danych (mogą być mniejsze niż produkcyjne). Używa **Circle Sandbox API Keys**. Zmienne środowiskowe konfigurowane przez platformę hostingową (Vercel, AWS Secrets Manager, GCP Secret Manager). Deploymenty automatyczne po pushu do gałęzi stagingowej (CI/CD).
- **Production:**
 - Cel: Środowisko dostępne dla końcowych użytkowników.
 - Konfiguracja: Główna gałąź w Git (np. main lub master). Skalowalna, monitorowana infrastruktura (Vercel dla frontendu, AWS/GCP/Render dla backendu/DB). Używa **Circle Production API Keys** (wymaga weryfikacji konta Circle). Wdrożenia kontrolowane, często manualne lub po zatwierdzeniu (CI/CD). Włączone logowanie, monitoring, alertowanie.

Zarządzanie Zmiennymi Środowiskowymi:

- Używaj plików .env **tylko** lokalnie (i dodaj je do .gitignore).
- Na Staging/Production zmienne środowiskowe powinny być wstrzykiwane przez system CI/CD lub platformę hostingową w bezpieczny sposób. Nigdy nie umieszczaj sekretów bezpośrednio w kodzie ani w repozytorium Git.

To bardzo szczegółowy zarys. Każdy z tych punktów można by dalej rozwijać, ale daje to solidny fundament do rozpoczęcia prac nad TipJar. Czy któryś element chciałbyś omówić jeszcze dokładniej?